File permissions in Linux

Project description

In this project, I audited and updated file and directory access controls within a Linux environment to align with organizational security policies. Using the Linux command-line interface (CLI), I examined existing permission strings (`rwxrwxrwx`) for standard and hidden files, interpreted user, group, and other access levels, and modified permissions using commands like `chmod`. This process ensures data integrity and enforces the principle of least privilege by restricting unauthorized user access to sensitive system directories and files.

Check file and directory details

```
researcher2@74908b04be43:~$ cd projects
researcher2@74908b04be43:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Jul  8 15:36 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Jul  8 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Jul  8 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_t.txt
researcher2@74908b04be43:~/projects$
```

To inspect the current file and directory permissions, I navigated into the `projects` directory using the `cd projects` command. Once inside, I executed the `ls -l` command to list the contents in a long-listing format. This output exposed the file types, permissions strings, owner (`researcher2`), group assignment (`research_team`), file sizes, and modification dates for the `drafts` directory and the four project text files (`project_k.txt`, `project_m.txt`, `project_r.txt`, and `project_t.txt`). Reviewing this information allowed me to identify which files had overly permissive access configurations before applying modifications.

Describe the permissions string

The 10-character permissions string determines the type of file and who can access it.

- **File Type:** The first character indicates whether the item is a regular file (`-`) or a directory (`d`). For example, `drafts` starts with a `d`, while `project_k.txt` starts with a `-`.
- **User (Owner) Permissions:** Characters 2–4 define the read (`r`), write (`w`), and execute (`x`) permissions for the file's owner (`researcher2`).
- **Group Permissions:** Characters 5–7 define access for members of the group (`research_team`).
- **Other Permissions:** Characters 8–10 define access for all other users on the system.

For example, `project_m.txt` has a permissions string of `-rw-r-----`. This means it is a regular file where the owner has read and write permissions (`rw-`), the group has read-only access (`r--`), and other users have no access at all (`---`).

Change file permissions

```
researcher2@74908b04be43:~/projects$ chmod o-w project_k.txt
researcher2@74908b04be43:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 16:06 ..
-rw--w---- 1 researcher2 research_team   46 Jul  8 15:36 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Jul  8 15:36 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Jul  8 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_t.txt
researcher2@74908b04be43:~/projects$
```

To modify permissions and enforce the principle of least privilege, I used the `chmod` command to restrict write access on a file that was previously overly permissive. Specifically, I ran `chmod o-w project_k.txt` to remove write permissions (`-w`) from the "other" (public) user category for `project_k.txt`.Afterward, I executed `ls -la` to verify the update. The long listing confirmed that the permissions string for `project_k.txt` successfully changed from

`-rw-rw-rw-` (seen in the previous step) to `-rw-rw-r--`, ensuring that unauthorized public users can no longer modify its contents. Running this command with the `-a` flag also revealed a hidden file, `.project_x.txt`, within the working directory.

Change file permissions on a hidden file

```
researcher2@74908b04be43:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 16:06 ..
-rw--w---- 1 researcher2 research_team   46 Jul  8 15:36 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Jul  8 15:36 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Jul  8 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_t.txt
researcher2@74908b04be43:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@74908b04be43:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 16:06 ..
-r--r----- 1 researcher2 research_team   46 Jul  8 15:36 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Jul  8 15:36 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Jul  8 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_t.txt
```

In this step, I managed authorization permissions for the hidden file `.project_x.txt` to enforce tighter access control. Initially, the file's permissions were set to `-rw--w----`, which allowed the owner read/write access but gave the group write-only access.

To correct this security discrepancy, I executed the command `chmod u-w,g-w,g+r .project_x.txt`. This combined command simultaneously stripped write access (`-w`) from both the user and the group while granting read permissions (`+r`) to the group. Running `ls -la` afterward confirmed that the permission string successfully updated to `-r--r-----`, restricting the file to read-only access for both the owner and the group while preventing unauthorized modifications.

Change directory permissions

```
researcher2@74908b04be43:~/projects$ chmod g-x drafts
researcher2@74908b04be43:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Jul  8 16:06 ..
-r--r----- 1 researcher2 research_team   46 Jul  8 15:36 .project_x.txt
drwx------ 2 researcher2 research_team 4096 Jul  8 15:36 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Jul  8 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Jul  8 15:36 project_t.txt
```

To secure directory access, I modified the permissions of the `drafts` directory. Previously, its permissions string was `drwx--x---`, which granted execute permissions to the group, allowing group members to navigate into the directory.

I executed the command `chmod g-x drafts` to remove the execute permission (`-x`) from the group (`g`). I then ran `ls -la` to verify the changes. The updated permissions string for `drafts` is now `drwx------`, successfully blocking the group from accessing or traversing the directory and ensuring only the owner (`researcher2`) has access.

Summary

In this project, I successfully audited, restricted, and verified access control configurations on files and directories using the Linux command-line interface. By adhering to organizational security policies and enforcing the **principle of least privilege**, I applied targeted adjustments to existing access permissions. This included stripping public write access from standard files (chmod o-w), aligning a hidden file's read/write attributes to restrict user and group modifications (chmod u-w,g-w,g+r), and completely disabling group navigation access to a critical workspace directory (chmod g-x). These implementations mitigated potential threat vectors, secured unauthorized local access points, and successfully safeguarded critical system assets while preserving mandatory data access for legitimate operational functions.